

MangWalo — Technical Overview

A local-first neighborhood borrow & lend noticeboard. Flutter web PWA, on-device rule-engine AI, zero backend. Built for MAL Lab 1: Flutter & Foundations — and deployed as a real product.

Live	https://mangwalo.vercel.app
Source	https://github.com/rishika-pixeldust/mangwalo
Stack	Flutter (Material 3) · Dart ≥ 3.12 · Riverpod 3 · Hive CE · deployed as static web on Vercel
Size	~4,700 lines of Dart in <code>lib/</code> , 66 tests, no code generation

1. What it is

Residents of one Mumbai neighborhood post **offers** ("I can lend a drill") and **requests** ("I need a ladder for an afternoon"), browse the board, and track lending: who borrowed what, and when it is due. Overdue loans flag themselves and jump the feed. Everything — listings, photos, settings — lives in the browser's IndexedDB. There is no account, no server, and after the initial asset load the app makes **zero network requests**; even CanvasKit and fonts are bundled.

The product was built against four graded pillars, and each is engineered into the app rather than documented around it: product thinking (§3), local AI (§4), security/privacy (§5), accessibility (§6).

2. Architecture

Feature-first layout with visible layer boundaries. The dependency rule is `ui → application → domain ← data` ; nothing imports Hive outside `*/data/` (plus the `main.dart` bootstrap that opens the boxes).

```
lib/src/
├── core/           clock (injectable now()), constants, validation
                    (sanitizer, privacy scanner, validators), shared widgets
├── features/
│   ├── ai/        LocalAiService interface + RuleBasedListingAi + const keyword tables
│   ├── home/      HomeShell: dark docked nav + center "New listing" action
│   ├── listings/  domain / data / application / ui for the core loop
│   ├── onboarding/ first-launch neighborhood picker
│   ├── settings/  neighborhood, theme, sample data, reset-all
└── theme/         "Warm Ledger" design system (hand-mapped ColorSchemes)
```

State is a small Riverpod 3 graph, no codegen. Hive boxes are opened in `main()` before `runApp` and injected via `ProviderScope` overrides — no async-bootstrap races. If storage fails to open (e.g. blocked IndexedDB), `main()` swaps in an `InMemoryListingRepository` and the UI shows a "changes won't survive a refresh" banner instead of white-screening.

<code>listingBox / settingsBox</code>	<code>Provider<Box<String>></code> (overridden in main)
<code>listingRepositoryProvider</code>	<code>Provider<ListingRepository></code>
<code>listingsProvider</code>	<code>StreamProvider<List<Listing>></code> ← <code>repository.watchAll()</code>
<code>feedFilterProvider</code>	<code>Notifier<FeedFilter></code>
<code>filteredListingsProvider</code>	<code>pure FeedFilter.apply(): filter + sort</code>
<code>listingByIdProvider</code>	<code>Provider.family</code> — detail auto-updates on writes
<code>listingSuggestionProvider</code>	debounced AI bridge (450 ms, ≥12 chars, request-id guard)
<code>localAiServiceProvider</code>	<code>Provider<LocalAiService></code> ← the AI swap point
<code>settingsProvider</code>	<code>Notifier<AppSettings></code>
<code>nowProvider</code>	<code>Provider<DateTime Function()></code> — testable time

Feed sorting is deliberate product logic: **overdue first → due-soonest → most recently updated**, computed in a pure function (`FeedFilter.apply`) that is unit-tested with zero mocks.

2.1 The two seams (recorded in ADR-0001)

The app is local-first now and sync-ready by construction. Two interfaces are the documented change points:

```
abstract interface class ListingRepository {
  Future<List<Listing>> getAll();
  Stream<List<Listing>> watchAll(); // emits current list on listen, then on change
  Future<Listing?> getById(String id);
  Future<void> put(Listing listing); // upsert by id
  Future<void> putAll(List<Listing> listings);
  Future<void> delete(String id);
  Future<void> clear(); // backs "Reset all local data"
}

abstract interface class LocalAiService {
  Future<ListingSuggestion> suggest(AiSuggestionInput input);
  AiEngineInfo get engineInfo; // drives the in-app transparency note
}
```

A phase-2 synced backend (Supabase-style, local cache + remote) implements `ListingRepository` ; a real on-device model implements `LocalAiService` . Both swap in one line at their provider.

2.2 Persistence: Hive CE + hand-written JSON codec

Listings live in a `Box<String>` keyed by UUID, value = `jsonEncode(codec.toJsonMap(listing))` . **No TypeAdapters, no build_runner** — this sidesteps Hive's `Map<dynamic,dynamic>` cast pitfalls on web and makes schema evolution trivial: the codec embeds `"v": 1` , supplies defaults for missing keys, repairs impossible states (a `lentOut` row without a `dueDate`), and skips undecodable rows instead of crashing the feed. `dueDate` is stored as a date-only `yyyy-MM-dd` string so UTC round-trips can never shift the calendar day; timestamps are ISO-8601 UTC.

The domain model separates two orthogonal lifecycles that most designs conflate — the conversation (`saved → contacted → closed`) and the item (`available → lentOut(dueDate, borrowerName) → returned`) — with the invariant `dueDate != null & lentOut` enforced by the only three mutation helpers (`markLentOut` / `markReturned` / `markAvailable`) and re-checked by the codec.

3. Product core: return-date tracking

The personal product feature. Lending an item captures an optional borrower first name and a return date pre-filled from the AI's suggested duration. Due math is a pure function over **calendar days in local time** (`computeDueInfo`), so "due tomorrow at any hour" is always exactly 1 day — no time-of-day off-by-ones — and the clock is injected (`nowProvider`), so tests pin it. The My-items view aggregates lending into a hero stat card ("2 of 5 items out · 1 overdue") with a segmented meter.

4. Local AI: a deterministic rules engine behind a model-shaped interface

`RuleBasedListingAi` (pure Dart: no model, no network, no randomness, no clock) turns a free-text description — Hinglish welcome — into suggestions:

```
"bosch ka drill machine, thoda purana but works fine, weekends only"
→ title "Bosch Drill Machine" · category Tools & Repair
  · tags [Well-worn, Working] · duration "3 days – most repairs are quick"
```

Pipeline: normalize → tokenize (+ light singular stemming) → score categories against **291 const keywords across 10 categories** (phrase match = 3 pts via bigram/trigram lookup, single word = 1 pt, each entry counts at

most once — "drill drill drill" can't game it; ties break by enum order) → title from the longest matched-keyword run, prefixed with the earliest of 46 canonically-cased brands (Bosch, Prestige, Yonex, boAt...) and acronym-cased (NCERT, DSLR, TT...) → 38 condition phrases (incl. "thoda purana", "chalta hai") → a per-category loan-duration table. Confidence maps from the winning score (0 none / 1–2 low / 3–4 medium / ≥5 high).

Determinism is a tested contract: same input → field-identical output, and garbage input returns `ListingSuggestion.empty`, never a throw. In rubric terms the engine is simultaneously the primary implementation and the permanent offline fallback; the `LocalAiService` boundary is where a quantized on-device model would slot in without touching a call site — the UI's debounce layer (450 ms, min 12 chars, **monotonic request-id guard**) already assumes a slow async engine, so a model's latency can never clobber newer input.

UX contract: suggestions render as labeled chips and **never auto-fill**; each chip is an explicit action, applied state shown with an icon. A disclosure line — driven by `engineInfo`, so it updates if the engine is swapped — reads: "Suggestions are generated on-device and work offline. Nothing you type leaves this device."

5. Security & privacy: data minimization, enforced in code

Threat framing (see `docs/security-baseline.md`): with no network, the adversary is data leakage through careless input, storage, and shared devices.

- **PII detection while typing.** `PrivacyScanner` (pure, deterministic, separate from the AI feature) runs regex heuristics tuned for India: phone numbers (`+91 / 0` -prefixed, separator-tolerant, digit-boundary guarded so order IDs and prices don't match), Mumbai `400xxx` PINs, unit numbers ("flat no 402"), and a weak-signal score (wing-unit shorthand "B-402", floors, society vocabulary) that warns at ≥ 2 — one "society WhatsApp group" alone stays silent. Warnings are non-blocking, quote the exact matched text, and announce via a semantics live region.
- **Landmark-only locations, hard-enforced.** In the area field any privacy signal — or > 40 % digits — is a validation error, not a warning.
- **Sanitize-then-validate, twice.** `sanitize()` strips control characters and invisible Unicode (zero-width, bidi overrides, BOM — removed outright so words don't split), normalizes whitespace (multiline mode preserves the user's line breaks), and clamps length. Validators check the sanitized form — the exact string that will be stored — so normalization can never turn passing input into something the validator would have rejected (a phone number typed with double spaces, say). The repository re-sanitizes on write: defense in depth.
- **Photos.** Picked images are decoded, orientation-baked, downscaled to ≤ 900 px, and re-encoded as JPEG q78 — which strips EXIF, including GPS — then stored as base64 in Hive. Nothing leaves the device.
- **Data control.** Borrower names are first-name-only and cleared on return; sample data is `isDemo` -flagged, obviously fake, and separately removable; Settings has a confirmed **Reset all local data** that also clears in-memory UI state. The repo carries no secrets (grep-clean, and `.gitignore` blocks `*.env` / `*.pem` / `*.key` belt-and-braces).

6. Accessibility engineering

- Every listing card is **one merged semantics node** with a composed, meaningful label ("Offer: Cricket bat, Sports & Fitness, opposite Jogger's Park, lent out, borrowed by Rahul, due in 2 days.") — no fragment spam, and no hardcoded activation hint (the `button` flag lets each platform supply its own, localized one).
- AI suggestion chips are **activatable** by assistive tech (`Semantics(button, selected, onTap)`), not merely announced; new suggestion sets are announced once via `SemanticsService`, never per keystroke.
- Privacy warnings are live regions (WCAG 4.1.3): safety messages are spoken when they appear.

- States are never color-only (icon + text everywhere), touch targets are ≥ 48 px (`MaterialTapTargetSize.padded` + explicit minimums), and a widget test drives the feed at 200 % text scale and asserts nothing overflows. Both themes are contrast-checked.

7. Design system: "Warm Ledger"

Hand-mapped `ColorScheme` s (not seed-generated) for light and dark: warm cream canvas `#F3EEE6` , near-white 28 px-radius cards, one peach-orange accent `#F2793C` , charcoal ink, and a "night" tone (`#211D19`) mapped onto `inverseSurface` to power the docked bottom nav and pill CTAs. Typography is Plus Jakarta Sans, five weights bundled offline (OFL). The bottom bar is a custom `NightNavBar` (charcoal slab, top radius 28) with the orange center action half-docked over its edge via `FloatingActionButtonLocation.centerDocked` plus reserved top padding, so the middle destination stays clear and the whole button remains tappable. Full token spec: `docs/design-system.md` ; a paste-ready whole-app brief for design sessions: `docs/claude-design-brief.md` .

8. Testing & quality

66 tests, all pure-VM (no device):

Area	Tests	Notable assertions
AI engine	11	Hinglish $\rightarrow$ category/brand/tags, collisions ("casio keyboard" $\neq$ "keyboard"), determinism double-run, anti-gaming, garbage tolerance
Privacy/validators	27	Every regex positive and negative (model "R15", order IDs), landmark hard-errors, sanitize modes incl. zero-width/bidi removal
Codec + Hive	13	Full round-trip, timezone-safe dates, old-schema tolerance, impossible-state repair, corrupt-row skip — repository tests run against real Hive in a temp dir
Domain	11	Due-date boundaries (today/tomorrow/overdue), filter + sort order
Widgets	4	Merged card semantics, 200 % text scale without overflow, empty states

Beyond unit tests, a **golden-capture harness** (`tool/demo/`) renders nine real app screens at 3x phone resolution with real fonts and a pinned clock. It has triple duty: source frames for the preview gallery and the narrated demo video, and a pixel-level regression net — it caught two real layout bugs that release builds clipped silently (an unbounded `Center` inflating the nav bar to full screen; a 19 px row overflow). The demo video itself is fully reproducible: goldens $\rightarrow$ PIL slide composition $\rightarrow$ macOS `say` narration (Rishi, en-IN) $\rightarrow$ ffmpeg Ken-Burns assembly $\rightarrow$ `docs/media/mangwalo-demo.mp4` (~2:19).

During development the diff was also swept by a 25-agent adversarial review (correctness / security / accessibility / web pitfalls); all 19 confirmed findings were fixed — including a validate-before-sanitize phone-number bypass and screen-reader-dead suggestion chips.

9. Build & deploy

```
flutter pub get
flutter run -d chrome --web-port=8080 # fixed port keeps the IndexedDB origin stable
flutter test && flutter analyze
flutter build web --release --no-web-resources-cdn
cd build/web && vercel deploy --prod # → https://mangwalo.vercel.app
```

`--no-web-resources-cdn` bundles CanvasKit locally, so the deployed app has no runtime dependency on Google's CDN — consistent with the local-first claim. The PWA manifest ships install icons and a standalone display mode; `index.html` carries an instant inline splash removed on `flutter-first-frame`. Honesty note: offline reload (a caching service worker) is documented future work — the claim is "zero network after load in a session", verified with DevTools offline.

10. Known limitations / phase 2

- Single-device by design this phase: neighbors can't see each other's boards yet. The `ListingRepository` seam + versioned codec is the recorded migration path to a synced backend.
- The AI engine is keyword-based: English/Hinglish dictionary coverage, brittle to typos, honest about not being ML (`docs/local-ai-note.md`).
- No auth, chat, payments, or notifications — deliberately out of scope (`docs/product-slice.md`).